

TRIVIATREK

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology

In
Computer Science and Engineering
School of Engineering and Sciences

Submitted by

M.Sai Koushik Reddy	AP23110010697
A.Prraneth	AP23110010713
T.Karthik	AP23110011058
Agastya Sagar .M	AP23110011072



SRM University – AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240

[December, 2025]

Table of Contents

1. ***Abstract***
2. ***Introduction***
3. ***System Requirements***
4. ***Technology Stack***
5. ***System Analysis***
6. ***System Design***
7. ***Component Descriptions***
8. ***Screenshots & Output***
9. ***Conclusion***
10. ***References***
11. ***Appendix***

Abstract

This project report presents the design and development of *TriviaTrek*, a frontend-only interactive quiz-based learning system created using modern web technologies. The application provides a responsive, intuitive, and engaging user interface that allows users to explore various quiz categories, attempt topic-based questions, and receive instant feedback using locally stored or mock data. TriviaTrek emphasizes a *component-driven architecture*, consistent *UI/UX design principles*, and smooth interactive functionality implemented entirely on the client side without backend integration. This report comprehensively documents the system's structure, modular design, workflow, testing methodologies, and overall outcomes, demonstrating a systematic and effective approach to building scalable and user-centric frontend applications.

1. Introduction

1.1 Background Digital learning platforms and interactive educational tools have become increasingly essential in modern learning environments. Quiz-based applications, in particular, help learners improve recall, measure understanding, and engage with content more actively. *TriviaTrek* is designed as a frontend-only quiz learning system that simulates the core functionalities of a full-fledged educational platform while focusing primarily on user interface quality and interactive behavior. By leveraging modern frontend technologies, TriviaTrek delivers a smooth and engaging quiz experience without requiring backend integration. Many academic, prototype, or early-stage projects require functional user interfaces without the complexity of backend implementation. Learners often need systems that demonstrate *UI/UX design*, *component-driven development*, and *interactive logic* while still remaining simple enough for frontend-focused development. TriviaTrek addresses this need by providing a realistic quiz platform that highlights category browsing, quiz navigation, scoring, and feedback mechanisms using mock or locally stored data.

1.3 Objectives

- To design a responsive and accessible quiz-based user interface.
- To implement interactive components for category selection, quiz execution, and results visualization.
- To simulate quiz logic, scoring, and user progression using mock data structures.
- To demonstrate modern frontend architecture through reusable components, context/state management, and client-side routing.

1.4 Scope

Includes:

- User interface screens for categories, quizzes, and results.
- Navigation flow for quiz progression and feedback.

State management using contexts or hooks.

Handling of quiz questions and scoring through mock/static data.

Excludes:

Backend database integration or real-time data updates.

User authentication systems or persistent login.

Leaderboards, cloud storage, or external APIs.

2. System Requirements

2.1 Functional Requirements

Users should be able to browse quiz categories.

Users can view subtopics or quiz sets within each category.

A basic user interface for mock login or profile selection (optional).

A fully interactive quiz-taking module with question display, options, and scoring.

A results page showing score, accuracy, and feedback.

Page navigation implemented through a client-side router.

2.2 Non-Functional Requirements

Responsive design across desktop, tablet, and mobile devices.

Fast performance enabled by optimized React components and Vite bundling.

Smooth and intuitive navigation flow throughout the application.

High component reusability and maintainable code structure.

Consistent UI/UX styling using a defined design system.

2.3 Hardware Requirements

A computer with modern web browser.

2.4 Software Requirements

Code Editor (e.g., Visual Studio Code).

Node.js and npm for dependency management and local development.

Modern Web Browser (Chrome / Firefox / Edge).

Vite as the project bundler and development server.

Tailwind CSS for styling and responsive design.

3. Technology Stack

1. HTML – Structure of the application.
2. CSS / Tailwind CSS – Styling and responsive layout.
3. JavaScript (ES6+) – Logic for interactivity.
4. React.js – Component-based UI library.

Tools Used

5. Vite (fast dev server)
6. ESLint (linting)
7. PostCSS (styling processor)

4. System Analysis

System analysis identifies user needs and defines how the proposed system will fulfill them. For **TriviaTrek**, the objective is to enhance the learning and quiz-taking experience through an intuitive, interactive, and user-friendly frontend interface. The system is designed to simplify category exploration, quiz participation, and performance review using a clean architecture and engaging UI elements, ensuring an efficient and enjoyable learning process for users.

4.1 Existing System In many traditional learning platforms and quiz applications, users often face challenges such as limited interactivity, outdated interfaces, and unclear navigation flows. These limitations reduce engagement, hinder effective learning, and make the overall quiz-taking experience less enjoyable. TriviaTrek addresses these issues by providing a modern, visually appealing, and highly interactive frontend system that enhances user motivation and simplifies the process of exploring and attempting quizzes.

4.2 Proposed System The proposed system delivers a clean, modern, and responsive user interface where users can effortlessly browse quiz categories, view detailed quiz sets, and participate in interactive assessments. It enhances user engagement by providing instant feedback, smooth transitions, and intuitive navigation. Built entirely using React and Tailwind CSS, the system ensures fast performance, eliminates unnecessary complexity, and offers a seamless quiz-taking experience through a fully frontend-based approach.

4.3 Advantages

□ ***User-FriendlyInterface:***

TriviaTrek provides an intuitive and visually appealing interface that allows learners to easily navigate through categories, quizzes, and results.

□ ***QuickNavigation:***

The application offers smooth and efficient transitions between pages, ensuring a fast and uninterrupted quiz-taking experience.

□ ***ModernResponsiveDesign:***

Built with Tailwind CSS and React, the system adapts seamlessly to desktops, tablets, and mobile devices, delivering a consistent and responsive user experience.

5. System Design

5.1 Navigation Flow

The navigation flow of ***TriviaTrek*** is structured to ensure a smooth and intuitive user experience. Each page in the frontend plays a specific role in guiding the learner through categories, subtopics, quizzes, and results. ***Landing*** → ***Home*** → ***Categories*** → ***Subtopics*** → ***Quiz*** → ***Results*** → ***Stats*** → ***Home*** Additional routes include: ***Login*** / ***Register*** (for mock authentication)
Learning Page (topic overview-style page)

- **About Page** (application description)

Navigation is fully handled through **React Router**, enabling fast transitions without page reloads.

5.2 Component Hierarchy

1. App.jsx

Responsibilities:

Acts as the root of the application.

Defines and manages all primary routes.

Wraps the application with global providers such as:

- o AuthContext
- o QuizContext
- o ThemeContext

Contains:

Navbar.jsx (persistent)

All pages rendered through React Router

2. Core Components

These components define reusable UI and logic across the system.

Navbar.jsx

Displays navigation links (Home, Categories, Login, Stats, etc.)

Visible on all main pages

CategoryCard.jsx

Renders individual quiz categories

Used in Home.jsx, Learning.jsx, SubtopicSelection.jsx

QuestionCard.jsx

Displays a quiz question and its options

Handles user selection and communicates answer state to QuizContext

ProgressBar.jsx

Shows quiz progress visually based on the current question index

Timer.jsx

Optional countdown component for timed quizzes

ProtectedRoute.jsx

Restricts certain pages (like results or stats) unless the user has completed a quiz or is authenticated (mock)

5.3 Contexts

1) *AuthContext.jsx*

Manages mock authentication (login/register)

Stores user session state

2) *QuizContext.jsx*

Stores selected category, subtopic, questions

Tracks user score, current question index, results

3) *ThemeContext.jsx*

Manages application theme (light/dark mode)

5.4 Data Layer

Stored under */src/data*

1. *quizCategories.js*

Contains metadata about each category (id, title, icon, description)

2. *quizData.js*

Main dataset containing questions, options, answers per category/subtopic

3. *quizData.backup.js*

Additional backup dataset

5.5 Pages

1. Landing.jsx

Initial introduction page

Highlights features and CTA (Get Started, Explore Categories)

2. Home.jsx

Displays all available quiz categories

Uses CategoryCard.jsx repeatedly

3. SubtopicSelection.jsx

Allows user to choose quiz subtopics for the selected category

Reads data from quizData.js

4. Quiz.jsx

Core quiz engine

Uses:

- o QuestionCard.jsx
- o Timer.jsx
- o ProgressBar.jsx

5. Results.jsx

Displays:

- o Score
- o Accuracy
- o Correct/incorrect questions

Uses values stored in QuizContext

6. Stats.jsx

Shows past performance (if stored)

Useful for mock analytics

7. Learning.jsx

Provides descriptive information about topics/subtopics (optional educational view)

8. Login.jsx

Simple mock authentication UI

9. About.jsx

Provides project information and credits

5.6 Application Entry Files

1. main.jsx

Bootstraps the React application

Wraps App.jsx inside strict mode

Injects context providers

2. index.css / App.css

- Global styles and Tailwind setups

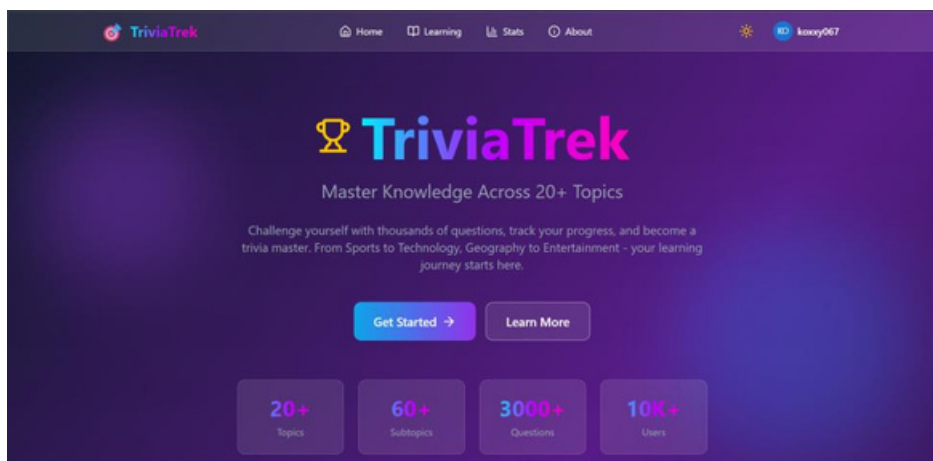
6. Component Descriptions

- Home Page
 - o Introduces the TriviaTrek platform and its purpose.
 - o Displays featured quiz categories or trending topics.
 - o Includes banners, highlights, or call-to-action buttons such as ***Start Learning*** or ***Explore Categories***.
- Categories Page
 - o Lists all available quiz categories (e.g., Sports, Science, Movies).
 - o Uses the ***CategoryCard*** component to render each category.
 - o Allows users to select a category to view related subtopics.
- Subtopic Selection Page
 - o Displays subtopics or quiz sets under the chosen category.
 - o Prepares the user before entering the quiz module
 - o Includes a selecting button
- Quiz Page
 - o Shows one question at a time using the ***QuestionCard*** component.
 - o Displays a timer using the ***Timer*** component.No backend database interaction
 - o Tracks quiz progress visually using the ***ProgressBar*** component.
 - o Uses ***QuizContext*** to manage answers, scoring, and current question index.
- Result Page
 - o Displays the user's final score, accuracy percentage, and performance summary.
 - o Shows correct vs. incorrect answers for review.
 - o Provides options to retry the quiz or return to categories.

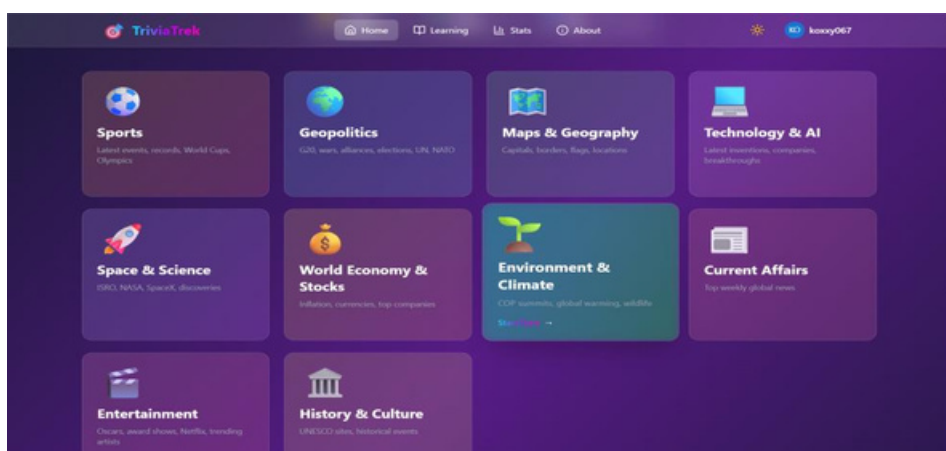
- o Uses data stored within *QuizContext*.
- Login & Registration Pages
 - o Provide UI screens for mock authentication
 - o Handle basic frontend validation for user inputs.
 - o Uses *AuthContext* to manage login state and user session.
 - o No backend authentication is performed—entirely frontend-based.

7. Screenshots & Output

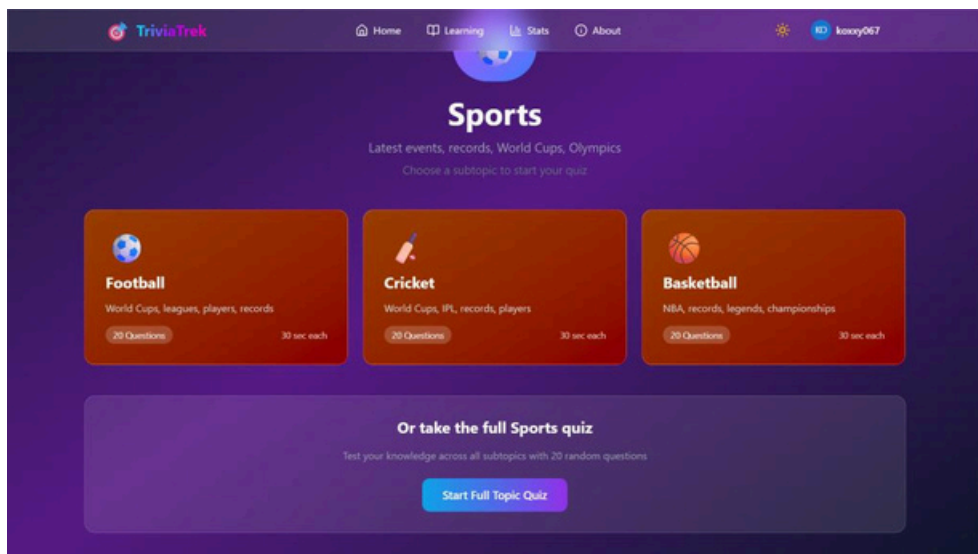
7.1 Home Page



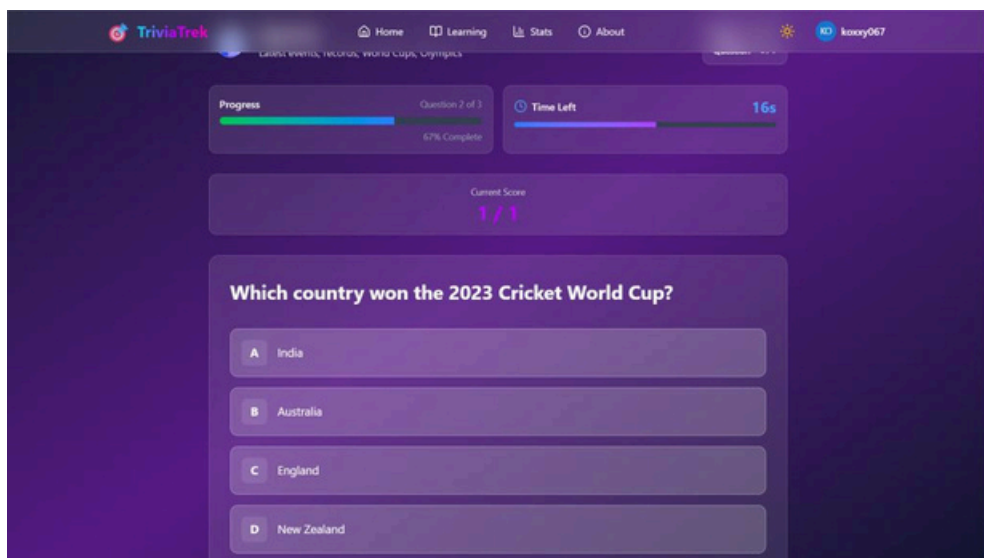
7.2 Categories Page



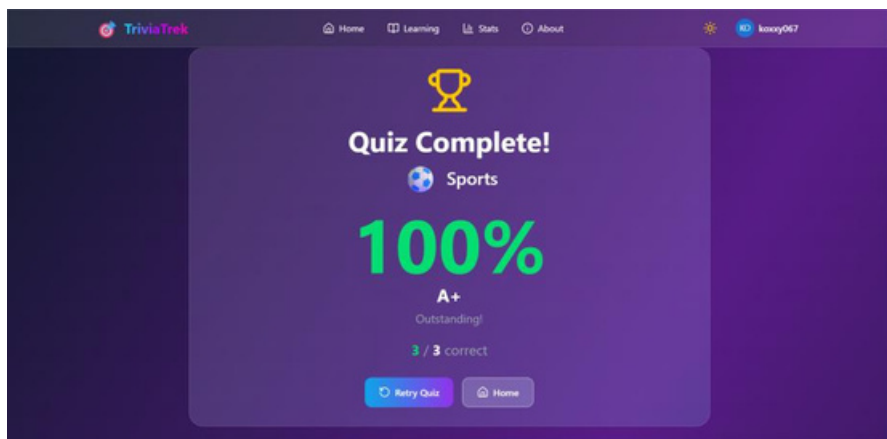
7.3 Subtopic Selection Page



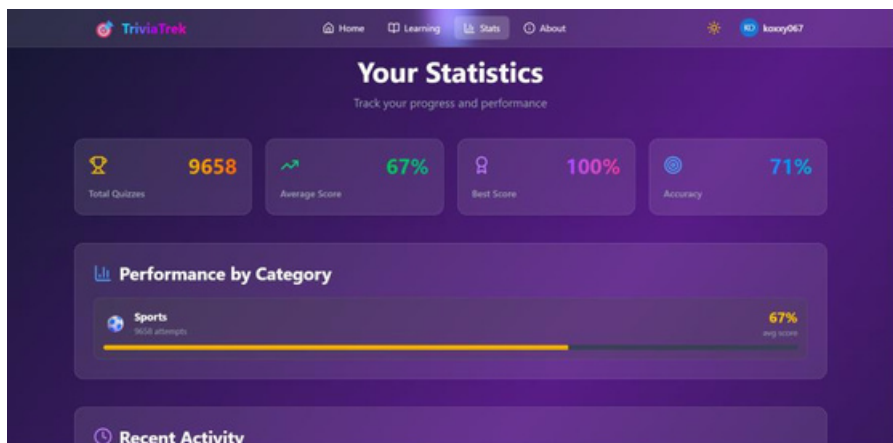
7.4 Quiz Pages



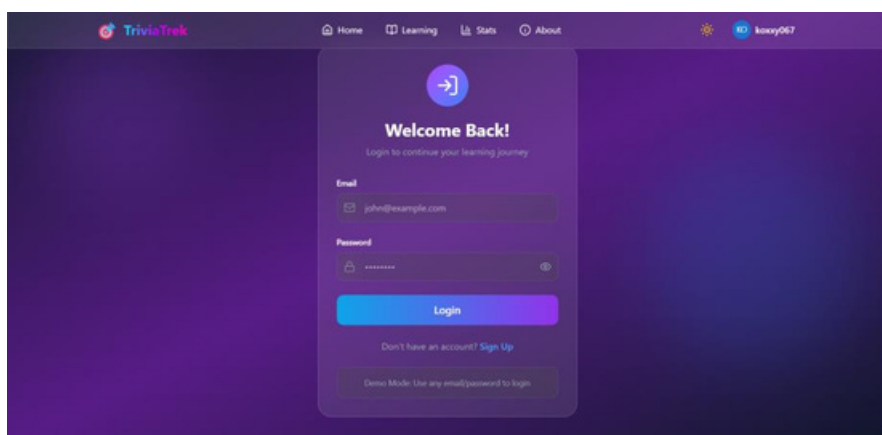
7.5 Results Page



7.6 Stats Page (Mock Analytics)



7.7 Login & Register Pages



8. Conclusion

TriviaTrek successfully demonstrates the design and development of a modern, responsive, and component-based quiz learning platform. Although implemented entirely on the frontend, the system delivers a high level of interactivity through dynamic question rendering, real-time scoring, and smooth navigation. By leveraging *React*, *Tailwind CSS*, and the *Context API*, the application achieves clean state management, reusable UI components, and an efficient architecture that resembles real-world educational platforms. Overall, TriviaTrek serves as a strong example of how frontend technologies can be used to build an engaging, scalable, and user-centric learning experience without requiring backend integration.

9. References

React Documentation

Tailwind CSS Docs

MDN Web Docs

Vite Documentation

10. Appendix

10.1 Git repositories link:

TRIVIATREK